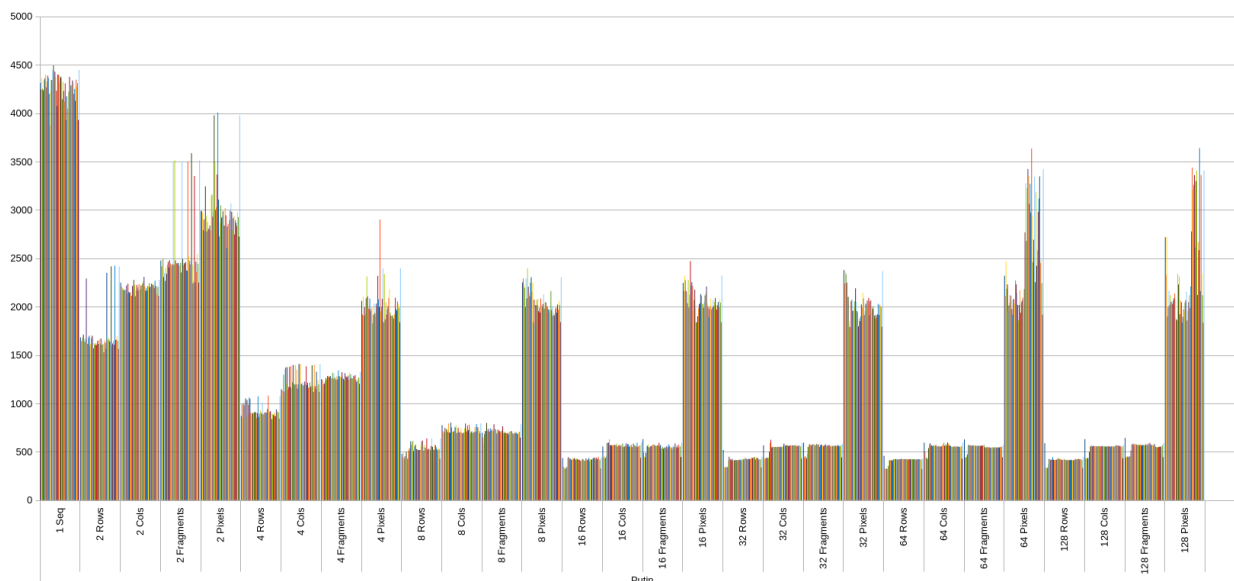




Ref (1): total_results.png в корне репозитория и выше ^^^^^^

Ref (2): benches_results.xlsx, также графики внутри файла для наглядности для ROWS и COLS

Условия эксперимента

ОС Linux, дистрибутив Manjaro 24.1.1, процессор Intel® Core™ i7-12650H, объем оперативной памяти 16 Гиб.

Метрика: время работы алгоритма, в миллисекундах.

Исследовательские вопросы:

1. Как увеличение числа потоков влияет на скорость свертки.
2. Какой вид доступа к данным (по строкам, по столбцам, по фрагментам, по пикселям) позволяет сильнее ускорить процесс свертки?

Неединичное число потоков и параллельная реализация алгоритма свертки действительно существенно (до 10 раз) снижают скорость работы алгоритма по любой из представленных оценок: среднее, медианное, 2-е минимальное и максимальное время работы алгоритма.

При этом можно заметить, что при достижении отметки в 16 потоков, рост производительности прекращается, и более того, в случае с фрагментарной обработкой, начинается снижение производительности при увеличении числа потоков до 128. Это вызвано ростом накладных расходов на создание программных потоков при поточном числе физических.

Построчный доступ к данным показал наибольшую эффективность, позволяя достичь более чем 10-кратного ускорения в сравнении с последовательной реализацией. Доступ к данным по колонкам и фрагментарно приводит к примерно (с разницей в считанные проценты при числе потоков больше 4) одинаковому росту скорости обработки данных. Наименьшую эффективность из представленных демонстрирует попиксельный доступ, обеспечивающий кратно меньший прирост производительности.